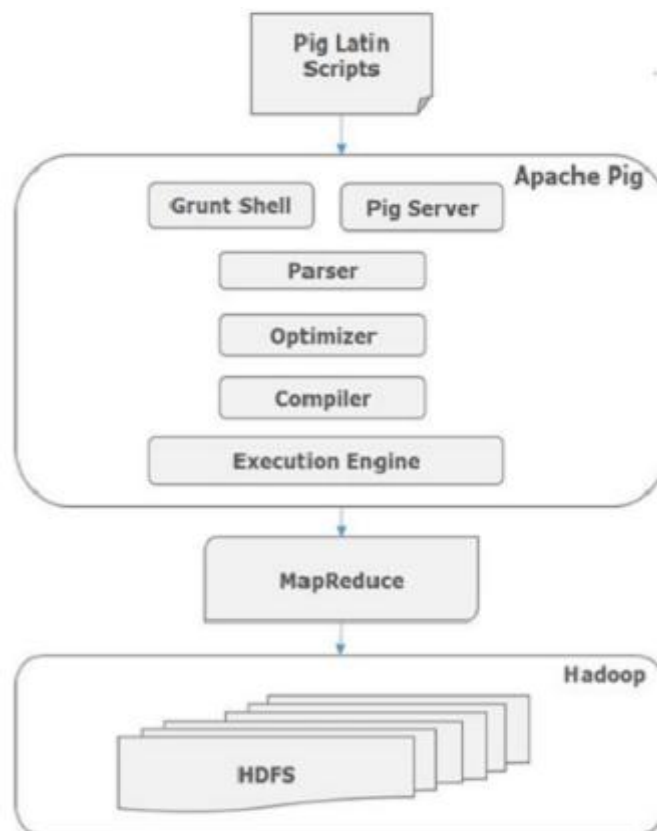


Department of Computer Science and Engineering

III/II CSE - R23 Regulations
BIG DATA ANALYTICS

Apache Pig Architecture – Detailed Explanation



1. Introduction to Apache Pig

Apache Pig is a high-level platform used for analyzing large datasets in the **Hadoop ecosystem**. It provides a scripting language called **Pig Latin**, which simplifies writing **MapReduce programs**.

Instead of writing complex Java MapReduce programs, developers can write simple Pig scripts that automatically get converted into MapReduce jobs.

Key idea:

Pig Latin Script → Converted into MapReduce Jobs → Executed on Hadoop Cluster.

Components of Apache Pig Architecture

The architecture of Apache Pig mainly consists of the following components:

Pig Latin Script

Parser

Logical Plan

Optimizer

Compiler

Execution Engine

Hadoop / MapReduce

HDFS Storage

1. Pig Latin Script

Pig Latin is the **data flow language** used in Apache Pig.

Users write scripts using Pig Latin commands to perform operations such as:

Load data

Filter data

Group data

Join datasets

Store results

Example

```
data = LOAD 'students.txt' USING PigStorage(',')  
      AS (id:int, name:chararray, marks:int);
```

```
filtered_data = FILTER data BY marks > 60;
```

```
STORE filtered_data INTO 'result';
```

This script loads data, filters students with marks > 60, and stores the result.

2. Parser

The **Parser** checks whether the Pig Latin script is **syntactically correct**.

Functions of Parser

Validates Pig Latin syntax

Performs **type checking**

Creates a **logical plan**

Ensures field names and data types are correct

If errors are found, the parser stops execution.

3. Logical Plan

The **Logical Plan** represents the sequence of operations defined in the Pig script.

It is a **data flow graph** that describes how data will be processed.

Example Flow

LOAD → FILTER → GROUP → STORE

This plan shows what operations need to be performed, but **not how they will run physically**.

4. Optimizer

The **Optimizer** improves the logical plan to make execution faster and more efficient.

Optimization Techniques

Projection pushdown

Filter pushdown

Join optimization

Reordering operations

Combining operations

Example:

Filtering data early reduces the amount of data processed later.

5. Compiler

The **Compiler** converts the optimized logical plan into a **physical plan**.

The physical plan contains actual execution steps like:

Map tasks

Reduce tasks

Data transfer between nodes

These are translated into **MapReduce jobs**.

6. Execution Engine

The **Execution Engine** executes the MapReduce jobs.

It communicates with Hadoop components such as:

JobTracker / Resource Manager

TaskTracker / Node Manager

Responsibilities:

Submit jobs to Hadoop

Monitor execution

Handle failures

Collect results

7. Hadoop MapReduce

The Pig execution engine submits jobs to the **Hadoop MapReduce framework**.

MapReduce performs:

Map Phase

Processes input data in parallel.

Reduce Phase

Aggregates intermediate results.

Example operations:

Group

Join

Aggregation

8. HDFS (Hadoop Distributed File System)

All data in Pig is stored and retrieved from **HDFS**.

HDFS provides:

Distributed storage

Fault tolerance

High scalability

Pig reads input data from HDFS and writes results back to HDFS.

Overall Workflow of Apache Pig Architecture

The overall data flow works as follows:

User Pig Script



Parser



Logical Plan



Optimizer



Compiler



Physical Plan



Execution Engine



MapReduce Jobs



Hadoop Cluster



HDFS Output

Modes of Apache Pig Execution

Pig can run in two modes:

1. Local Mode

Runs on a **single machine**

Uses local file system

Useful for **testing and debugging**

Example:

`pig -x local script.pig`

2. MapReduce Mode

Runs on **Hadoop cluster**

Uses **HDFS**

Suitable for **big data processing**

Example:

pig script.pig

Advantages of Apache Pig

Simple scripting language (Pig Latin)

Reduces need for Java MapReduce programming

Supports large-scale data processing

Handles semi-structured and unstructured data

Extensible using **User Defined Functions (UDFs)**

Summary

Component	Function
Pig Latin Script	User writes data processing commands
Parser	Checks syntax and creates logical plan
Logical Plan	Represents sequence of operations
Optimizer	Improves execution efficiency
Compiler	Converts logical plan to physical plan
Execution Engine	Executes MapReduce jobs
Hadoop MapReduce	Processes data in parallel
HDFS	Stores input and output data